Controle Adaptativo de Bitrate Server-Side em Plataforma Open Source de Cloud Gaming

César de Paula Morais[1]

[1]Departamento de Ciência da Computação – Universidade Federal de Minas Gerais (UFMG)
Orientador: Prof. Dr. Luiz Filipe Menezes Vieira
Tipo de Pesquisa: Mista

cesar.morais@dcc.ufmg.br

1. Introdução

1.1. Contextualização

O *cloud gaming* — modalidade em que o processamento gráfico de jogos é executado em um servidor remoto e transmitido em tempo real para o dispositivo do jogador — deixou de ser uma promessa e se tornou realidade comercial. Serviços como NVIDIA GeForce Now, Microsoft xCloud e PlayStation Plus permitem que usuários joguem títulos de alta exigência em dispositivos modestos, eliminando a necessidade de hardware dedicado. O mercado global de cloud gaming foi avaliado em aproximadamente US$ 1.5 bilhões em 2025 [Pagano et al. 2025] e projeta crescimento contínuo na próxima década.

No entanto, esses serviços apresentam limitações estruturais relevantes: são centralizados, pagos por assinatura e sujeitos a descontinuação unilateral. O encerramento abrupto do Google Stadia em 2023 ilustrou essa fragilidade, deixando usuários sem acesso a conteúdo pelo qual haviam pago. A dependência de infraestrutura proprietária levanta questões sobre propriedade, privacidade e continuidade do acesso.

Essa fragilidade impulsionou um movimento alternativo: jogadores com hardware potente passaram a hospedar seus próprios servidores de streaming, transmitindo jogos para outros dispositivos dentro de casa ou via internet, sem depender de terceiros. O **Sunshine** é o principal servidor de cloud gaming open source em atividade. Compatível com o protocolo NVIDIA GameStream e com o cliente **Moonlight** — disponível em Android, iOS, PC e smart TVs —, o projeto conta com centenas de contribuidores ativos e é amplamente utilizado pela comunidade de self-hosting. Uma contribuição técnica ao Sunshine beneficia diretamente usuários reais e, se aceita pelos mantenedores, vai a produção.

1.2. Trabalho Desenvolvido Anteriormente

Este projeto tem como ponto de partida a pesquisa desenvolvida no POC I, cujo tema era a **detecção de ataques de False Data Injection (FDI) em pelotões veiculares usando Reinforcement Learning**. O trabalho anterior estabeleceu um ambiente de simulação completo utilizando Plexe, SUMO e OMNeT++ para modelagem realística de pelotões veiculares; implementação controlada de ataques FDI via modificações no código-fonte do framework; geração de dataset balanceado com aproximadamente 9.000 amostras contendo cenários com e sem ataques; e um modelo de detecção baseado em PPO (*Proximal Policy Optimization*) treinado offline.

Os resultados preliminares, embora demonstrem a viabilidade da abordagem, revelaram limitações significativas: **acurácia de 52,6%, sensibilidade de 69,1% e especificidade de apenas 35,6%**. Esses indicadores são insuficientes para aplicações práticas em sistemas de segurança veicular, onde falhas na detecção podem ter consequências críticas. Mais importante, a análise dos resultados não apontou hipótese clara de melhoria: a especificidade baixa sugeria uma limitação estrutural da abordagem no domínio — não apenas de hiperparâmetros — sem direção promissora para um POC II no mesmo tema. Os objetivos originais do POC II (implementação robusta das abordagens selecionadas e validação experimental) foram, portanto, descartados junto com o tema.

Durante a exploração de projetos open source ativos como alternativa, identificou-se uma oportunidade técnica concreta no Sunshine: a ausência de controle adaptativo de bitrate no servidor, problema com impacto direto na qualidade de experiência dos usuários e lacuna não coberta pela literatura existente.

1.3. O Problema a ser Investigado

O **bitrate de codificação** — quantidade de bits por segundo utilizada para comprimir cada frame — é o principal parâmetro de qualidade em cloud gaming. Bitrates mais altos produzem imagens mais fiéis; bitrates mais baixos introduzem artefatos visuais como blocagem e desfoque.

O Sunshine negocia o bitrate uma única vez, durante o handshake inicial da sessão (protocolo RTSP), e mantém esse valor **fixo durante toda a transmissão**, independentemente das condições da rede. Em redes instáveis — Wi-Fi, conexões com perda de pacotes, ou internet residencial sob carga —, o encoder continua transmitindo no bitrate máximo negociado. Quando a perda de pacotes excede a capacidade de recuperação da correção de erros (FEC), a tela do jogador congela. O cliente solicita um frame de recuperação (IDR); o servidor o produz, mas não reduz o bitrate. O ciclo se repete indefinidamente.

O Sunshine já recebe sinais contínuos de degradação do cliente — contagem de pacotes perdidos e solicitações de recuperação — mas nenhum é utilizado para ajustar o bitrate. **A infraestrutura de feedback já existe; o controlador não.** O detalhamento técnico deste mecanismo é apresentado na seção 2.2.

Este trabalho investiga duas perguntas em sequência. A **pergunta principal**: um controlador ABR server-side, operando sobre sinais observáveis pelo servidor sem sincronização de relógio com o cliente, produz métricas de estabilidade melhores do que o comportamento fixo atual? A **pergunta exploratória**: é possível ir além do estado da arte explorando sinais exclusivos do posicionamento server-side — como a taxa de variação da perda de pacotes ou o intervalo de chegada das mensagens de controle — em comparação com abordagens client-side como o ABRAGame?

1.4. Objetivos

Objetivo Geral: projetar, implementar e avaliar empiricamente um controlador adaptativo de bitrate server-side para o Sunshine, contribuído ao repositório oficial do projeto por meio de Pull Requests.

Objetivos Específicos:

- Instrumentar o Sunshine com exportação estruturada de métricas de sessão para arquivo CSV, provendo infraestrutura de observabilidade atualmente inexistente na plataforma (Pull Request #1);
- Configurar um testbed de emulação de rede via `tc netem` replicando cenários reais de degradação, e coletar dados de baseline do comportamento atual do Sunshine sob cada cenário;
- Engajar os mantenedores do repositório oficial por meio de issue pública, apresentando o problema, os dados do baseline e o esboço do algoritmo antes da implementação;
- Implementar um controlador adaptativo de bitrate (PR #2), avaliá-lo comparativamente ao baseline de bitrate fixo, e documentar os resultados independentemente do desfecho.

2. Referencial Teórico

2.1. Cloud Gaming: Arquitetura e Restrições

O cloud gaming interativo distingue-se do streaming de vídeo convencional por uma restrição fundamental: o conteúdo não existe até o jogador agir. Em plataformas de vídeo sob demanda (VoD) como Netflix, o mesmo conteúdo é pré-codificado em múltiplas qualidades antes de qualquer transmissão — formando uma *bitrate ladder* — e o servidor seleciona o arquivo adequado à condição da rede. Em cloud gaming, cada frame é gerado em resposta ao input do jogador e deve ser capturado, codificado e transmitido dentro de uma janela de aproximadamente 16 ms (a 60 fps). É impossível pré-codificar o que ainda não existe.

Essa restrição impõe consequências em cascata. Os encoders operam em presets otimizados para velocidade em detrimento de eficiência de compressão — o NVENC da NVIDIA, por exemplo, é configurado nos modos de menor latência, completando a codificação em 2–8 ms por frame. A análise preditiva de complexidade de cena — possível em VoD — é inviável: uma explosão de partículas e um menu estático chegam ao encoder na mesma janela, sem aviso prévio.

Os requisitos de latência reforçam essas restrições. Pesquisas sobre percepção de qualidade em cloud gaming identificam 100 ms como limiar de tolerância à latência para jogos de ação, e RTTs acima de 70 ms como prejudiciais à jogabilidade em gêneros competitivos [BALDOVINO, 2022]. Buffers de reprodução — que em VoD absorvem variações de rede com segmentos de 10–30 segundos armazenados antecipadamente — são proibidos no cloud gaming: cada milissegundo de buffer adicionado é um milissegundo de latência entre o input do jogador e a resposta visual.

2.2. Pipeline Técnico do Sunshine

O Sunshine implementa o pipeline de streaming em três estágios. Na **captura**, frames do display são lidos via backends específicos por plataforma — NvFBC (NVIDIA), KMS/DRM ou portal Wayland no Linux, DXGI no Windows. Na **codificação**, os frames são comprimidos via GPU pelos backends NVENC (NVIDIA), VAAPI (Intel/AMD) ou AMF (AMD), com suporte a software como fallback. Na **transmissão**, os frames codificados são enviados ao cliente via UDP com proteção de erros por FEC (*Forward Error*

Correction), que adiciona pacotes redundantes para recuperação de perdas moderadas sem retransmissão.

O bitrate de codificação — parâmetro central deste trabalho — é negociado durante o handshake inicial do protocolo RTSP: o cliente declara sua largura de banda máxima disponível, o servidor subtrai a sobrecarga do FEC, do áudio e dos cabeçalhos de protocolo, e o resultado é configurado no encoder. Esse valor nunca é modificado durante a sessão. O servidor recebe continuamente sinais de degradação do cliente (`IDX_LOSS_STATS` com contagem de pacotes perdidos; `IDX_REQUEST_IDR_FRAME` com solicitações de frame de recuperação), mas esses sinais são apenas registrados em log — nenhum mecanismo de controle de bitrate os consome.

2.3. Trabalhos Relacionados

Hong et al. (2021) [Hong et al.] propuseram uma plataforma open source de cloud gaming com suporte a adaptação de bitrate, demonstrando viabilidade técnica do problema em ambiente controlado. O trabalho, no entanto, não aprofunda o design do controlador nem avalia seu comportamento em cenários sistemáticos de degradação de rede.

Baldovino (2022) [Baldovino 2022], em revisão sistemática, identificou apenas 33 trabalhos relevantes publicados entre 2017 e 2022 em IEEE Xplore e ACM. Entre os trabalhos identificados, destacam-se o **Themis** — controlador baseado em aprendizado por reforço para particionamento de recursos que reduziu atraso de resposta em 17–36% — e o **DeepGame** — sistema de codificação baseado em visão computacional. Ambos operam em contextos de infraestrutura de data center, distantes do cenário de self-hosting que o Sunshine representa.

ABRAGame [Armendariz and Joskowicz 2025] é o trabalho mais próximo do aqui proposto. Implementa um controlador rule-based inteiramente client-side, sem modificações no servidor. O algoritmo acumula latência de rede em uma janela deslizante de 500 ms; se a latência acumulada supera o limiar equivalente a $\sim$80 ms sustentados, o bitrate é reduzido para `estBW - 1,5 Mbps`. A recuperação é linear: `+0,5 Mbps` a cada 20 segundos de estabilidade. O sistema foi testado em plataforma proprietária com emulação de rede via NetEm em 6 cenários de degradação. Resultados: 100% de detecção de degradações, 10,3% de falsos positivos, 85,9% de cancelamento efetivo de frame loss, tempo médio de recuperação de 4,5 segundos. A principal limitação do ABRAGame para este trabalho é que sua operação client-side exige medição de latência de via única, o que requer sincronização de relógio entre cliente e servidor — e opera em plataforma proprietária não reproduzível pela comunidade.

GCC (*Google Congestion Control*) [Holmer et al. 2016], algoritmo de controle de taxa do padrão **WebRTC**, é uma referência industrial diretamente relevante. O WebRTC transmite áudio e vídeo via UDP sem buffering — o mesmo modelo do cloud gaming. O GCC combina dois sinais complementares: **perda de pacotes** (reativo — detecta congestionamento depois que já ocorreu) e **gradiente de delay** (preditivo — detecta aumento na variação do delay de entrega antes que pacotes sejam descartados). Essa combinação é o que torna o GCC eficaz em redes instáveis, e motiva a investigação de se um sinal preditivo análogo pode ser construído server-side no Sunshine.

2.4. Lacuna Identificada

Uma busca complementar conduzida neste trabalho em IEEE Xplore, ACM Digital Library e arXiv com os termos *"adaptive bitrate game streaming"*, *"server-side ABR cloud gaming"* e *"reinforcement learning bitrate game streaming"*, filtrando publicações de 2023 a 2025, confirmou que a área permanece pouco explorada. A busca não encontrou nenhum trabalho que implemente controle adaptativo de bitrate server-side em plataforma open source de cloud gaming. O ABRAGame [Armendariz and Joskowicz 2025], trabalho mais recente e tecnicamente mais próximo, é client-side, exige sincronização de relógio e opera em plataforma proprietária. O GCC [Holmer et al. 2016], embora resolva o problema de controle de taxa em tempo real sobre UDP, não foi projetado nem avaliado no contexto de cloud gaming.

3. Metodologia

3.1. Abordagem Geral

A metodologia proposta é dividida em cinco passos sequenciais, onde cada etapa produz insumos necessários para a seguinte. O projeto parte da premissa de que **os dados do baseline devem guiar o design do controlador** — não premissas a priori — e que a validação com a comunidade deve preceder a implementação. Três questões norteiam cada decisão: o que é factível no escopo de um semestre? O que ainda não foi feito na literatura? O que tem impacto real para usuários da plataforma?

3.2. Passo 1 — Exportação de Métricas de Sessão (PR #1)

Instrumentar `src/stream.cpp` para exportar métricas de sessão para arquivo CSV a cada intervalo de reporte do `IDX_LOSS_STATS`. As métricas exportadas incluem: timestamp, bitrate configurado, contagem de pacotes perdidos, taxa de IDR requests e largura de banda estimada. Esta infraestrutura de observabilidade é pré-requisito para todos os passos subsequentes e tem valor independente para a comunidade — atualmente não há forma estruturada de inspecionar o comportamento de uma sessão Sunshine. A frequência de reporte do `IDX_LOSS_STATS` será medida empiricamente nesta etapa, pois determina a resolução temporal da janela deslizante do controlador.

3.3. Passo 2 — Configuração do Testbed

Configurar emulação de rede via `tc netem` no host Linux, replicando os 6 cenários de degradação utilizados pelo ABRAGame:

Cenário	Descrição
A	Queda abrupta + recuperação rápida
B	Queda gradual + recuperação rápida
C	Queda lenta + recuperação rápida
D	Flutuações intermitentes
E	Queda longa + recuperação abrupta
F	Início baixo + melhora tardia

Duração: 240 segundos por cenário. Cliente: Moonlight Android (smartphone). Servidor: PC com NVIDIA RTX 4070 Super, Fedora Linux, captura via X11. A replicação dos cenários do ABRAGame permite comparação direta dos resultados com o trabalho de referência.

3.4. Passo 3 — Coleta do Baseline

Executar o Sunshine com bitrate fixo em todos os 6 cenários. Coletar os CSVs gerados pelo PR #1. Quantificar o comportamento atual da plataforma — esses dados cumprem dois papéis: (1) embasam o argumento da issue pública (Passo 4) com evidência empírica; (2) informam os critérios de sucesso do controlador, substituindo thresholds arbitrários por referências derivadas dos dados reais.

3.5. Passo 4 — Issue no GitHub do Sunshine

Abrir issue de discussão no repositório oficial do Sunshine apresentando: descrição do problema, dados coletados no Passo 3, esboço do algoritmo proposto e perguntas sobre preferências arquiteturais dos mantenedores. O objetivo é validar a abordagem antes de escrever código, coletar feedback técnico da comunidade e estabelecer registro público da proposta — práticas padrão de contribuição a projetos open source ativos.

3.6. Passo 5 — Implementação e Avaliação (PR #2)

Design do algoritmo: o controlador executa a cada mensagem `IDX_LOSS_STATS` recebida do cliente — o mesmo evento que hoje o Sunshine apenas loga. A cada execução, agrega taxa de perda de pacotes e taxa de requisições de IDR sobre uma janela deslizante, produzindo um sinal composto de congestionamento.

O controlador tem duas decisões possíveis. Se o sinal supera um limiar de congestionamento, o bitrate é reduzido agressivamente para a largura de banda estimada menos uma margem de segurança, respeitando um mínimo configurável — e um timer de estabilidade é reiniciado. Se o sinal está abaixo do limiar e o timer de estabilidade atingiu 20 segundos sem interrupção, o bitrate é incrementado conservadoramente por um passo fixo, respeitando um máximo configurável. Se nenhuma condição for satisfeita, o bitrate permanece inalterado. A reconfiguração do encoder é chamada apenas quando há mudança. Os parâmetros — margem de redução, passo de incremento, limiar de congestionamento e tamanho da janela deslizante — serão calibrados a partir dos dados do baseline. O controlador será ativável via configuração (`bitrate_mode = adaptive` em `sunshine.conf`), sem impacto para usuários que não o habilitarem.

Uma segunda variante será investigada após a análise do baseline, condicionada ao que os dados revelarem. A motivação para essa investigação é a tensão central do design: os sinais primários do controlador (taxa de perda de pacotes, requisições de IDR) são **reativos** — chegam ao servidor *depois* que o congestionamento já causou dano. O ABRAGame, por contraste, usa latência acumulada, que é **preditivo**: detecta a fila de pacotes enchendo *antes* do overflow. O GCC demonstrou que combinar um sinal reativo com um preditivo é o que torna um controlador robusto em redes instáveis. A questão de pesquisa não é apenas "o controlador melhora o baseline?" — é "sinais server-side são suficientemente preditivos para *evitar* frame loss, ou apenas para *reagir* a ele?" Um resultado negativo nessa frente seria, em si, uma contribuição para o controle preditivo em cloud gaming server-side.

Métricas de avaliação:

Métrica	Descrição	Referência (ABRAGame)
Taxa de cancelamento de frame loss	% de eventos de degradação evitados	85,9%
Tempo de recuperação	Segundos até retorno ao bitrate nominal	4,5 s
Taxa de falsos positivos	Reduções desnecessárias em rede estável	10,3%
Variância do bitrate	Estabilidade do controlador ao longo do tempo	—

Table 1. Métricas propostas de avaliação

3.7. Riscos e Mitigações

Os principais riscos do projeto são de natureza técnica e acadêmica. No lado técnico, a frequência de reporte do `IDX_LOSS_STATS` pode ser insuficiente para a resolução temporal desejada — risco mitigado pela medição empírica no Passo 1, que permitirá ajustar a janela deslizante antes de qualquer implementação. No lado acadêmico, os sinais server-side disponíveis podem se mostrar insuficientemente preditivos para antecipar congestionamento antes que frame loss ocorra — resultado que, se confirmado, será documentado como contribuição: delimita o que é possível sem sincronização de relógio. Quanto à aceitação pela comunidade, a abertura prévia de issue (Passo 4) alinha expectativas com os mantenedores antes da implementação; independentemente de merge, o código permanecerá publicado no fork como contribuição reproduzível.

3.8. Uso de Inteligência Artificial e Gestão de Conhecimento

Este projeto adota uma metodologia de **documentação contínua assistida por IA**, com transparência explícita sobre o que foi delegado à ferramenta versus o que é de autoria própria.

O **Claude Code** (CLI da Anthropic) foi utilizado como assistente de pesquisa e desenvolvimento ao longo do projeto: análise de código-fonte do Sunshine, estruturação de documentos acadêmicos e suporte à implementação em C++. O desenvolvimento conceitual, as decisões de design e as conclusões são de autoria do pesquisador.

O conhecimento acumulado durante o projeto é organizado em uma **base de conhecimento em Obsidian** — ferramenta de notas com suporte a *wikilinks* e visualização em grafo. A base contém arquivos cobrindo arquitetura do Sunshine, protocolos de streaming, codecs, trabalhos relacionados e decisões de design, todos interconectados por referências bidirecionais. O **grafo de conhecimento** resultante — ilustrado na Figura 1 — funciona como artefato metodológico: registra a topologia da pesquisa e torna visível o caminho intelectual percorrido, não apenas o resultado final — e será atualizado no final do projeto.

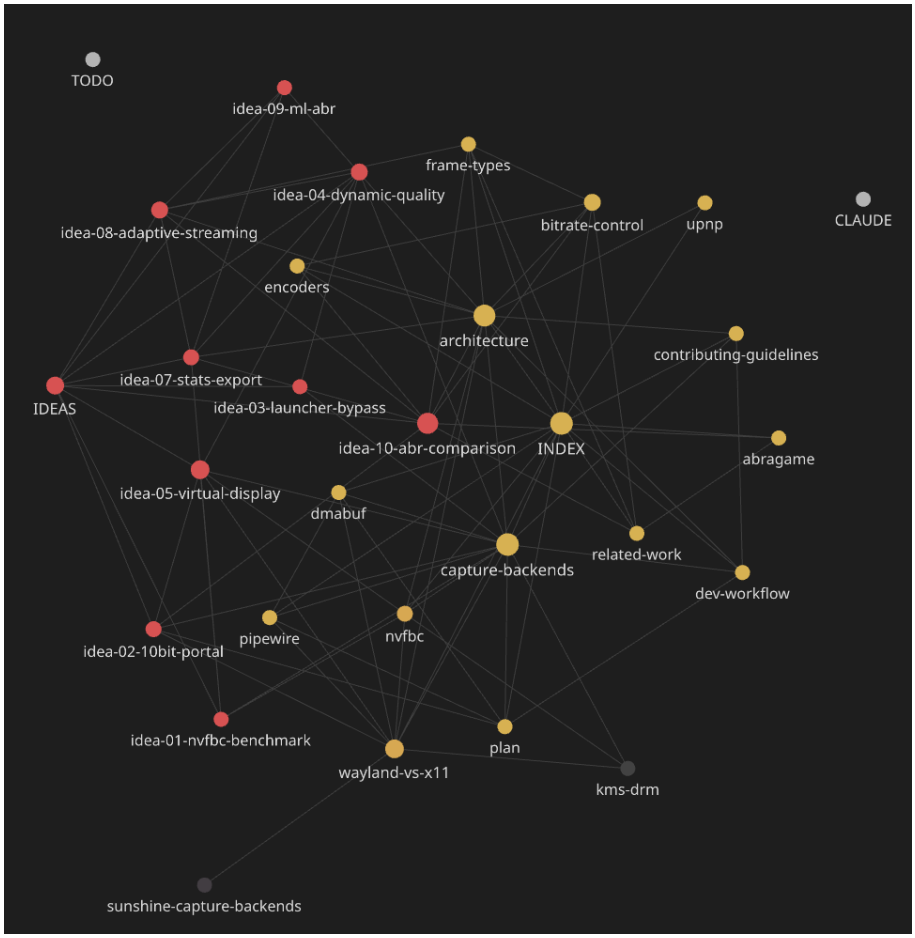


Figure 1. Grafo de conhecimento do projeto no Obsidian

As ideias exploradas ao longo do projeto são rastreadas em um diretório dedicado, documentando o processo de seleção do tema e das abordagens adotadas.

4. Resultados Esperados

Ao final do projeto, espera-se a produção de dois Pull Requests submetidos ao repositório oficial do Sunshine: o PR #1 (exportação de métricas de sessão), com valor independente como ferramenta de diagnóstico para a comunidade, e o PR #2 (controlador adaptativo de bitrate), contribuição técnica central deste trabalho. Independentemente de aceitação pelos mantenedores, ambos permanecerão publicados no fork do autor como código aberto e reproduzível.

A avaliação comparativa entre o baseline de bitrate fixo e o controlador adaptativo nos 6 cenários de degradação constituirá o resultado científico principal. Os critérios quantitativos de sucesso serão definidos após a coleta do baseline — os dados do comportamento atual do Sunshine sob degradação informarão o que constitui melhora prática relevante. Como referência qualitativa, espera-se redução na taxa de frame loss e no tempo de recuperação nos cenários de degradação, com taxa de falsos positivos aceitável em redes estáveis. O ABRAGame serve como ponto de comparação para calibrar as expectativas, reconhecendo que a operação server-side impõe restrições adicionais nos sinais disponíveis.

Não se pode descartar, a priori, que os sinais server-side disponíveis (perda de pacotes, taxa de IDR requests) sejam insuficientemente preditivos para antecipar congestionamento antes que frame loss ocorra. Esse resultado negativo seria, em si, uma contribuição científica: delimitaria empiricamente as possibilidades do design server-side sem sincronização de relógio e motivaria trabalho futuro com mecanismos alternativos de feedback.

Além dos entregáveis técnicos, espera-se o aprimoramento das competências do pesquisador no ciclo completo de contribuição a projetos open source ativos em C++17 — da identificação do problema à revisão por pares por engenheiros da comunidade —, bem como na aplicação prática de conceitos de sistemas, redes e controle em um produto com usuários reais.

5. Etapas e Cronograma

Semana	Período	Atividades Previstas
0 (pré)	13/03 – 10/04	Desenvolvimento da proposta; revisão bibliográfica complementar
—	**10/04**	**Entrega da proposta (Moodle)**
1–2	13/04 – 26/04	PR #1: exportação de métricas para CSV
3–4	28/04 – 10/05	Configuração do testbed (`tc netem`); coleta e análise do baseline nos 6 cenários
5	11/05 – 17/05	Abertura de issue no GitHub do Sunshine; início da implementação do controlador
6	18/05 – 24/05	Implementação do controlador adaptativo (PR #2); testes iniciais; preparação do pitch
—	**25/05**	**Apresentação parcial — Pitch Vídeo**
7–8	26/05 – 08/06	Testes comparativos nos 6 cenários; iteração com feedback da issue e do pitch
9	09/06 – 15/06	Análise dos resultados; início da escrita do relatório
10	16/06 – 21/06	Finalização do relatório; preparação do pôster
—	**21/06**	**Entrega do pôster para impressão pelo departamento**
11	22/06 – 26/06	Revisão final do relatório
—	**26/06**	**Apresentação final — Pôster (presencial)**
12	27/06 – 29/06	Ajustes finais
—	**29/06**	**Entrega do relatório final (PDF via Moodle)**
—	**03/07**	**Entrega na plataforma DCC (POC II)**

Table 2. Cronograma previsto para realização das atividades do projeto, com marcos de entrega destacados em amarelo.

References

Armendariz, M. and Joskowicz, J. (2025). ABRAGame: Automatic bit rate adjustment for cloud gaming. *Multimedia Tools and Applications*.

Baldovino, B. (2022). An overview of the networking issues of cloud gaming: A literature review. *JINITA: Journal of Informatics and Networking Technology and Application*.

Holmer, C., Lundin, H., Carlucci, G., De Cicco, L., and Bråthen, S. (2016). A Google congestion control algorithm for real-time communication. Internet draft, IETF. draft-ietf-rmcat-gcc-02.

Hong, H.-J. et al. Enabling adaptive cloud gaming in an open-source cloud gaming platform. *IEEE Transactions on Circuits and Systems for Video Technology*.

Pagano, E., Paizanis, G., Joniškis, P., Danielsen, N., Smart, F., Modi, S., and Anderson, H. (2025). Video gaming report 2026: The next era of growth.